

Лабораторная работа №2
Исследование протокола TCP и алгоритма управления
очередью RED

Шубнякова Дарья НКНбд-01-22

Содержание

1	Цель работы	3
2	Задание	3
3	Теоретическое введение	3
4	Выполнение лабораторной работы	4
5	Выводы	9

1 Цель работы

Исследовать протокол TCP и алгоритм управления очередью RED. Требуется разработать сценарий, реализующий модель, построить в Xgraph график изменения TCP-окна, график изменения длины очереди и средней длины очереди.

2 Задание

– Измените в модели на узле s1 тип протокола TCP с Reno на NewReno, затем на Vegas. Сравните и поясните результаты. – Внесите изменения при отображении окон с графиками (измените цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

3 Теоретическое введение

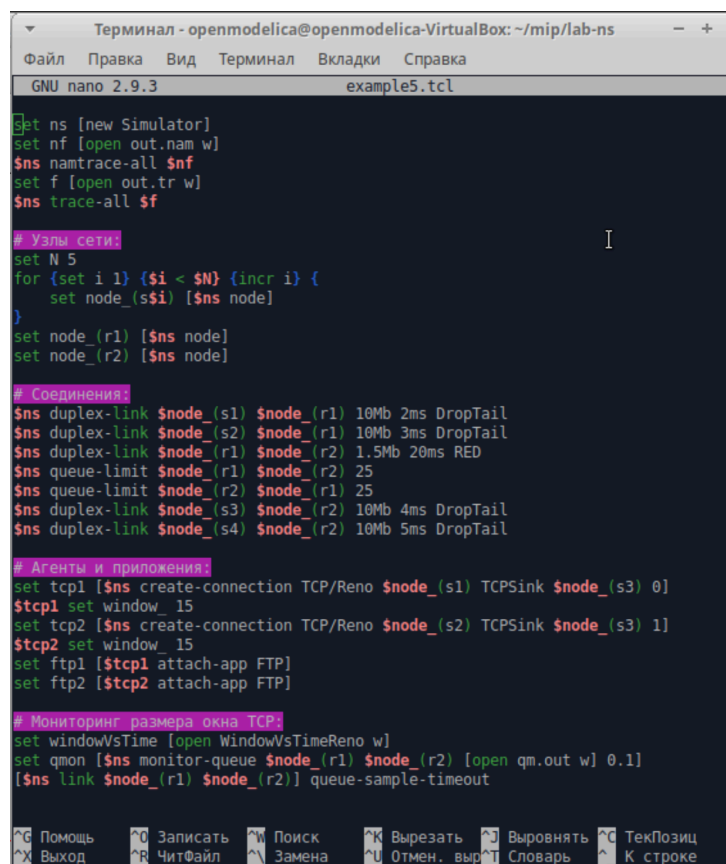
Протокол управления передачей (Transmission Control Protocol, TCP) имеет средства управления потоком и коррекции ошибок, ориентирован на установление соединения. TCP Reno: - медленный старт (Slow-Start); - контроль перегрузки (Congestion Avoidance); - быстрый повтор передачи (Fast Retransmit); - процедура быстрого восстановления (Fast Recovery); - метод оценки длительности цикла передачи (Round Trip Time, RTT), используемой для установки таймера повторной передачи (Retransmission TimeOut, RTO).

Алгоритм RED (Random Early Detection) – это метод управления перегрузками в сетевых маршрутизаторах, который заблаговременно отбрасывает пакеты, чтобы предотвратить полную перегрузку очереди. В отличие от простых методов, RED обнаруживает и реагирует на потенциальную перегрузку на ранних стадиях, случайным образом снижая количество передава-

емых пакетов, что помогает стабилизировать работу сети.

4 Выполнение лабораторной работы

Создаем нужную нам модель с дуплексными узлами. Берем алгоритм из лабораторной работы(рис. 1).



```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3                                example5.tcl

set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set f [open out.tr w]
$ns trace-all $f

# Узлы сети:
set N 5
for {set i 1} {$i < $N} {incr i} {
    set node_($i) [$ns node]
}
set node_r1 [$ns node]
set node_r2 [$ns node]

# Соединения
$ns duplex-link $node_s1 $node_r1 10Mb 2ms DropTail
$ns duplex-link $node_s2 $node_r1 10Mb 3ms DropTail
$ns duplex-link $node_r1 $node_r2 1.5Mb 20ms RED
$ns queue-limit $node_r1 $node_r2 25
$ns queue-limit $node_r2 $node_r1 25
$ns duplex-link $node_s3 $node_r2 10Mb 4ms DropTail
$ns duplex-link $node_s4 $node_r2 10Mb 5ms DropTail

# Агенты и приложения
set tcp1 [$ns create-connection TCP/Reno $node_s1 TCPSink $node_s3 0]
$tcp1 set window 15
set tcp2 [$ns create-connection TCP/Reno $node_s2 TCPSink $node_s3 1]
$tcp2 set window 15
set ftp1 [$tcp1 attach-app FTP]
set ftp2 [$tcp2 attach-app FTP]

# Мониторинг размера окна TCP
set windowVsTime [open WindowVsTimeReno w]
set qmon [$ns monitor-queue $node_r1 $node_r2 [open qm.out w] 0.1]
$ns link $node_r1 $node_r2 queue-sample-timeout
```

Рисунок 1

Вот так будет выглядеть процедура финиш(рис. 2) (рис. 3).

```

# Проверяем finish:
proc finish {} {
    global ns nf f tchan_ windowVsTime

    $ns flush-trace
    close $f
    close $nf
    close $windowVsTime

    if { [info exists tchan_] } {
        close $tchan_
    }
}

```

Рисунок 2

```

# подключение кода AWK:
set awkCode {
    {
        if ($1 == "Q" && NF>2) {
            print $2, $3 >> "temp.q";
        }
        else if ($1 == "a" && NF>2) {
            print $2, $3 >> "temp.a";
        }
    }
}

set f [open temp.queue w]
puts $f "TitleText: red"
puts $f "Device: Postscript"

exec rm -f temp.q temp.a
exec touch temp.a temp.q

# выполнение кода AWK
catch {exec awk $awkCode all.q}

puts $f "\"queue"
catch {exec cat temp.q >@ $f}
puts $f "\n"
puts $f "\"ave_queue"
catch {exec cat temp.a >@ $f}
close $f

```

Рисунок 3

И пишем часть для отображения графиков в Xgraph(рис. 4).

```

# Запуск xgraph с графиками окна TCP и очереди:
exec xgraph -bb -tk -x time -t "TCP Reno Cwnd" WindowVsTimeReno &
exec xgraph -bb -tk -x time -y queue temp.queue &

# Запуск nam
exec nam out.nam &

exit 0
}

$ns run

```

Рисунок 4

Так будет выглядеть график динамики размера окна TCP Reno(рис. 5).

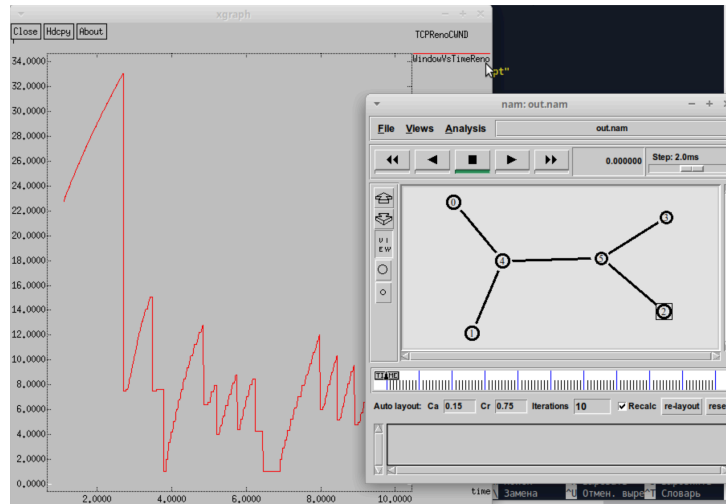


Рисунок 5

А здесь мы видим график динамики длины очереди и средней длины очереди(рис. 6).

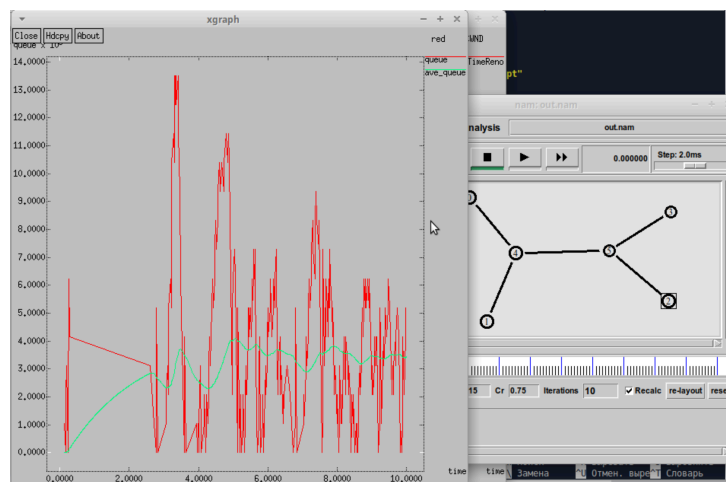


Рисунок 6

Меняем на алгоритм NewReno, важно соблюдать именно такое написание, как на скриншоте(рис. 7).

```
# Агенты и приложения
set tcp1 [$ns create-connection TCP/Newreno $node_(s1) TCPSink $node_($s2)
$tcp1 set window_ 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_($s3)
$tcp2 set window_ 15
set ftp1 [$tcp1 attach-app FTP]
set ftp2 [$tcp2 attach-app FTP]
```

Рисунок 7

Меняем цвет линий графика, фон графика, слегка меняем подписи осей, легенду и названия окон, чтобы показать освоение навыков в Xgraph(рис. 8).

```
set f [open temp.queue w]
puts $f "TitleText: RED"
puts $f "Device: Postscript"
puts $f "0.Color: Blue"
puts $f "1.Color: Orange"

exec rm -f temp.q temp.a
exec touch temp.q temp.a

# выполнение кода Awk
catch {exec awk $awkCode all.q}

puts $f "\"current_queue\""
catch {exec cat temp.q >@ $f}
puts $f "\n"
puts $f "\"average_queue\""
catch {exec cat temp.a >@ $f}
close $f

# Запуск xgraph с графиками окна TCP и очереди:
exec xgraph -bb -tk -x TIME -t "TCPNewRenoCwnd" WindowVsTimeNewReno -bg white
exec xgraph -bb -tk -x TIME -y QUEUE temp.queue -bg white &
```

Рисунок 8

Меняем так же цвет линии в графике динамики размера окна TCP(рис. 9).

```
# Мониторинг размера окна TCP
set windowVsTime [open WindowVsTimeNewReno w]
puts $windowVsTime "0.Color: Blue"
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1]
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout
```

Рисунок 9

Видим, что все изменения были применены, и оцениваем график TCP NewReno(рис. 10).

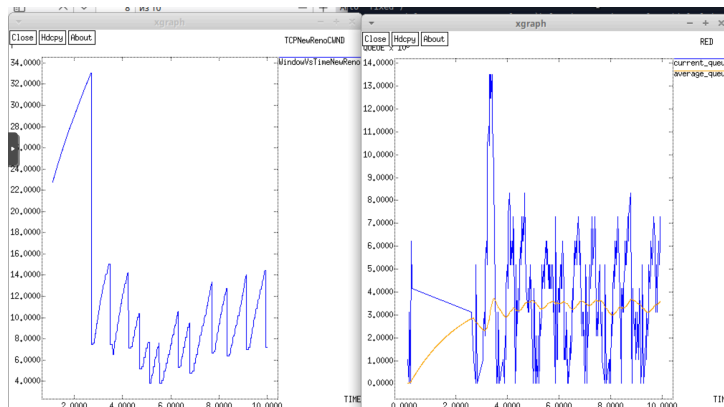


Рисунок 10

Меняем цвет линии на графике динамики размера окна TCP и на первом узле ставим TCP Vegas(рис. 11).

```
# Агенты и приложения
set tcp1 [$ns create-connection TCP/Vegas $node_(s1) TCPSink $node_(s2)]
$tcp1 set window 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3)]
$tcp2 set window 15
set ftp1 [$tcp1 attach-app FTP]
set ftp2 [$tcp2 attach-app FTP]

# Мониторинг размера окна TCP
set windowVsTime [open WindowVsTimeVegas w]
puts $windowVsTime "0.Color: Purple"
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1]
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout
```

Рисунок 11

И меняю тайтлы и цвета для графика средней и текущей очереди(рис. 12).


```

set f [open temp.queue w]
puts $f "TitleText: RED"
puts $f "Device: Postscript"
puts $f "0.Color: Pink"
puts $f "1.Color: Grey"

exec rm -f temp.q temp.a
exec touch temp.a temp.q

# выполнение кода Awk
catch {exec awk $awkCode all.q}

puts $f "\"current_queue"
catch {exec cat temp.q >@ $f}
puts $f "\n"
puts $f "\"average_queue"
catch {exec cat temp.a >@ $f}
close $f

# Запуск xgraph с графиками окна TCP и очереди
exec xgraph -bb -tk -x TIME -t "TCPVegasCWND" WindowVsTimeVegas -s
exec xgraph -bb -tk -x TIME -y QUEUE temp.queue -bg white &
# Запуск nam
exec nam out.nam &

```

Рисунок 12

Оцениваем работу при TCP Vegas(рис. 13).

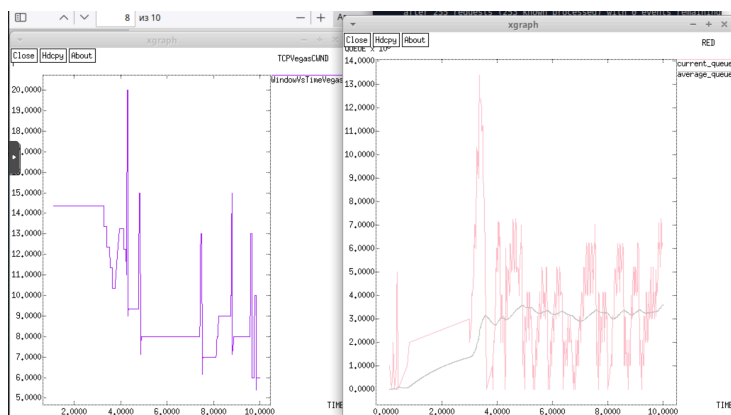


Рисунок 13

5 Выводы

Графики динамики размера окна у Reno и NewReno довольно похожи, второй действует чуть более плавно и нет таких сильных разбросов на протяжении работы. График динамики размера окна TCP Vegas очень ступенчатый,

перепады резкие, нет плавных линий. В сети с RED очередью: - Reno: постоянно “прощупывает” пределы пропускной способности - Vegas: находит оптимальный уровень и держится near него - При изменении условий Vegas делает шаг к новому оптимальному уровню

График средней и текущей очередей у NewReno и Reno опять же похожи, но текущая очередь у NewReno сразу начинает выравниваться после пика, а у Reno это происходит постепенно. График Vegas опять же в отличие от первых двух начинается с падения. TCP Reno/NewReno: - Текущая очередь: резкие пики и провалы, хаотичные колебания - Средняя очередь: менее изменчивая, но все равно волнистая - Причина: реагируют только на потери, постоянно “перескакивают” оптимальный уровень TCP Vegas: - Текущая очередь: более стабильная, плавные изменения - Средняя очередь: почти прямая линия near оптимального уровня - Причина: proactively регулирует окно на основе задержки, предотвращает переполнение очереди